

Curso: Tecnólogo em Análise e Desenvolvimento de Sistemas

Período: 2026/02

Disciplina: Programação Orientada a Objetos

Material: Introdução - Domínio, Classes e Objetos

0 PARADIGMA O.O. COM A LINGUAGEM JAVA

O desenvolvimento de software é extremamente amplo. Nesse mercado, existem diversas linguagens de programação, que seguem diferentes paradigmas. Um desses paradigmas é a Orientação a Objetos, que atualmente é o mais difundido entre todos. Isso acontece porque se trata de um padrão que tem evoluído muito, principalmente em questões voltadas para segurança e reaproveitamento de código, o que é muito importante no desenvolvimento de qualquer aplicação moderna.

A Programação Orientada a Objetos (POO) diz respeito a um padrão de desenvolvimento que é seguido por muitas linguagens, como C++, C# e Java. No Paradigma Orientado à Objetos, todo o sistema é organizado em um conjunto de entidades, chamadas **objetos**, que são representações do conceito de **classes**. O sistema é portanto representado por um *conjunto de objetos* e pelas *interações* que ocorrem entre eles.

O Paradigma OO é o que mais reflete os problemas atuais de Desenvolvimento de Sistemas. Linguagens orientada a objetos (OO) são projetadas para implementar diretamente a abordagem OO na solução dos problemas. Tal abordagem tornou-se uma ferramenta importante para solucionar muitos tipos de problemas através da simulação. Um programa OO consiste em objetos que enviam mensagens uns para os outros. Estes objetos no programa correspondem diretamente aos objetos atuais, tais como pessoas, máquinas, departamentos, documentos e assim por diante.

0.1 Vantagens do Paradigma OO

A programação orientada a objetos traz uma ideia muito interessante: a representação de cada elemento em termos de um objeto, ou classe. Esse tipo de representação procura aproximar o sistema que está sendo criado ao que é observado no mundo real, e um objeto contém características e ações, assim como vemos na realidade. Esse tipo de representação traz algumas vantagens muito interessantes para os desenvolvedores e também para o usuário da aplicação. Veremos algumas delas a seguir.

A reutilização de código é um dos principais requisitos no desenvolvimento de software atual. Com a complexidade dos sistemas cada vez maior, o tempo de desenvolvimento iria aumentar exponencialmente caso não fosse possível a reutilização. A orientação a objetos permite que haja uma reutilização do código criado, diminuindo o tempo de desenvolvimento, bem como o número de linhas de código. Isso é possível devido ao fato de que as linguagens de programação orientada a objetos trazem representações muito claras de cada um dos elementos, e esses elementos normalmente não são interdependentes. Essa independência entre as partes do software é o que permite que esse código seja reutilizado em outros sistemas no futuro.

Outra grande vantagem que o desenvolvimento orientado a objetos traz diz respeito a leitura e manutenção de código. Como a representação do sistema se aproxima muito do que vemos na vida real, o entendimento do sistema como um todo e de cada parte individualmente fica muito mais simples. Isso permite que a equipe de desenvolvimento não fique dependente de uma pessoa apenas, como acontecia com frequência em linguagens estruturadas como o C, por exemplo.

A criação de bibliotecas é outro ponto que é muito mais simples com a orientação a objetos. No caso das linguagens estruturadas, como o C, temos que as bibliotecas são coleções de procedimentos (ou funções) que podem ser reutilizadas. No caso da POO, entretanto, as bibliotecas trazem representações de classes, que são muito mais claras para permitirem a reutilização.

Entretanto, nem tudo é perfeição na programação orientada a objetos. A execução de uma aplicação orientada a objetos é mais lenta do que o que vemos na programação estruturada, por exemplo. Isso acontece devido à complexidade do modelo, que traz representações na forma de classes. Essas representações irão fazer com que a execução do programa tenha muitos desvios, diferente da execução sequencial da programação estruturada. Esse é o grande motivo por trás da preferência pela linguagem C em hardware limitado, como sistemas embarcados. Também é o motivo pelo qual a programação para sistemas móveis como o Google Android, embora em Java (linguagem orientada a objetos), seja feita o menos orientada a objetos possível.

No momento atual em que estamos, tecnologicamente essa execução mais lenta não é sentida. Isso significa que, em termos de desenvolvimento de sistemas modernos, a programação orientada a objetos é a mais recomendada devido as vantagens que foram apresentadas. Essas vantagens são derivadas do modelo de programação, que busca uma representação baseada no que vemos no mundo real.

A seguir serão apresentados os principais conceitos do Paradigma OO de Desenvolvimento.

0.2 Domínio da Aplicação

Um domínio é composto pelas entidades, informações e processos relacionados a um determinado contexto. Uma aplicação pode ser desenvolvida para automatizar ou tornar factível as tarefas de um domínio. Portanto, uma aplicação é basicamente o “reflexo” de um domínio. Para exemplificar, suponha que estamos interessados em desenvolver uma aplicação para facilitar as tarefas do cotidiano de um banco. Podemos identificar clientes, funcionários, agências e contas como entidades desse domínio. Assim como podemos identificar as informações e os processos relacionados a essas entidades. Cada aplicação OO está relacionada à um domínio, dentro do qual podemos identificar entidades que serão modeladas e programadas, em **classes** e **objetos**. A Figura 1 ilustra um exemplo de domínio para um sistema bancário, onde identificamos entidades e representamos as mesmas na aplicação OO.

As entidades identificadas no domínio devem ser representadas de alguma forma dentro da aplicação correspondente. Nas aplicações orientadas a objetos, as entidades são representadas por **objetos**. Uma **aplicação orientada a objetos** é composta por objetos. Em geral, um objeto representa uma entidade do domínio. Para exemplificar, suponha que no domínio de um determinado banco exista um cliente chamado João. Dentro de uma aplicação orientada a objetos correspondente a esse domínio, deve existir um objeto para representar esse cliente. Suponha que algumas informações do cliente João como **nome**, **data de nascimento** e **sexo** são importantes para o banco. Já que esses dados são relevantes para o domínio, o objeto que representa esse cliente deve possuir essas informações. Esses dados são armazenados nos **atributos** do objeto que representa o João.

Um **atributo** é uma *variável que pertence a um objeto*. Os dados de um objeto são armazenados nos seus atributos. O próprio objeto deve realizar operações de consulta ou alteração dos valores de seus atributos. Essas operações são definidas nos **métodos** do objeto. Os **métodos** também são utilizados para possibilitar interações entre os objetos de uma aplicação. Por exemplo, quando um cliente requisita um saque através de um caixa eletrônico do banco, o objeto que representa o caixa eletrônico deve interagir com o objeto que representa a conta do cliente. As tarefas que um objeto pode realizar são definidas pelos seus métodos. Um objeto é composto por atributos e métodos.

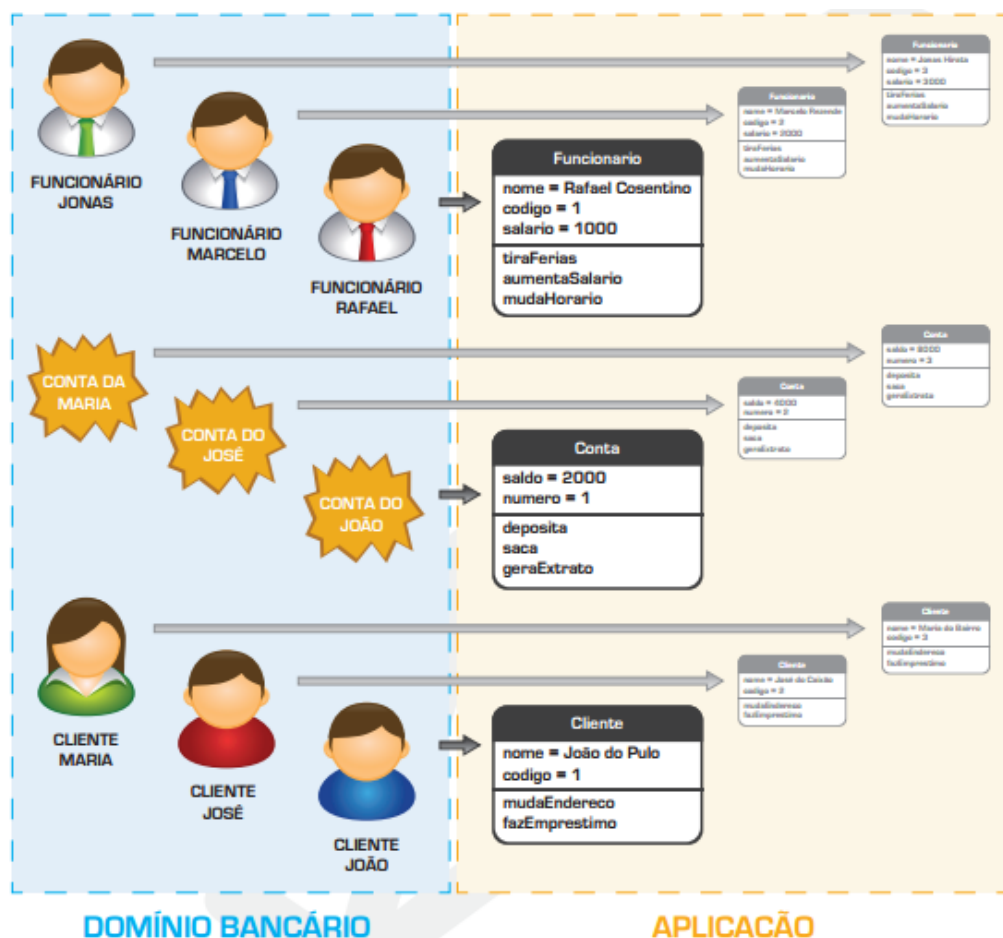


Figura 1. Entidades de um domínio de Sistema Bancário e representação na Aplicação OO.

0.3 Classes e Objetos

A definição de classes e seus inter-relacionamentos é o principal resultado da etapa de projeto de software. Em geral, esse resultado é expresso em termos de alguma linguagem de modelagem, tal como UML (*Unified Modeling Language*). A UML surgiu como padrão para a **modelagem** de sistemas OO, representando o comportamento do sistema sob diversos aspectos (**diagramas UML**).

Uma classe é um **modelo** para a geração de objetos. Através da definição de uma classe, descreve-se que propriedades — ou atributos — o objeto terá. Portanto, antes de um objeto ser criado, devemos definir quais serão os seus atributos e métodos. Essa definição é realizada através de uma classe elaborada por um programador. A partir de uma classe, podemos construir objetos na memória do computador que executa a nossa aplicação.

Podemos representar uma classe através de diagramas UML. O diagrama UML de uma classe é composto pelo **nome** da mesma e pelos **atributos** e **métodos** que ela define. Todos os objetos criados a partir da classe Conta terão os atributos e métodos mostrados no diagrama UML. Os valores dos atributos de dois objetos criados a partir da classe Conta podem ser diferentes. A Figura 2 ilustra um exemplo de representação de uma classe Conta em UML

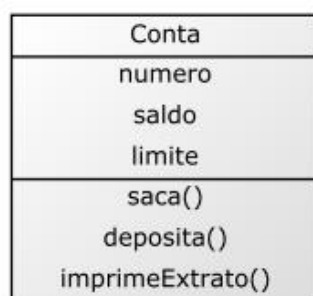


Figura 2. Representação da classe Conta em UML.

Uma classe é portanto, um modelo a partir do qual serão gerados diversos objetos. Cada objeto terá o mesmo conjunto de operações da classe, ou seja os mesmos métodos. Entretanto, cada objeto é diferente pelo conjunto de atributos que o mesmo possui. Por exemplo, para a classe conta podemos criar os seguintes objetos identificados na Figura 3 a seguir:

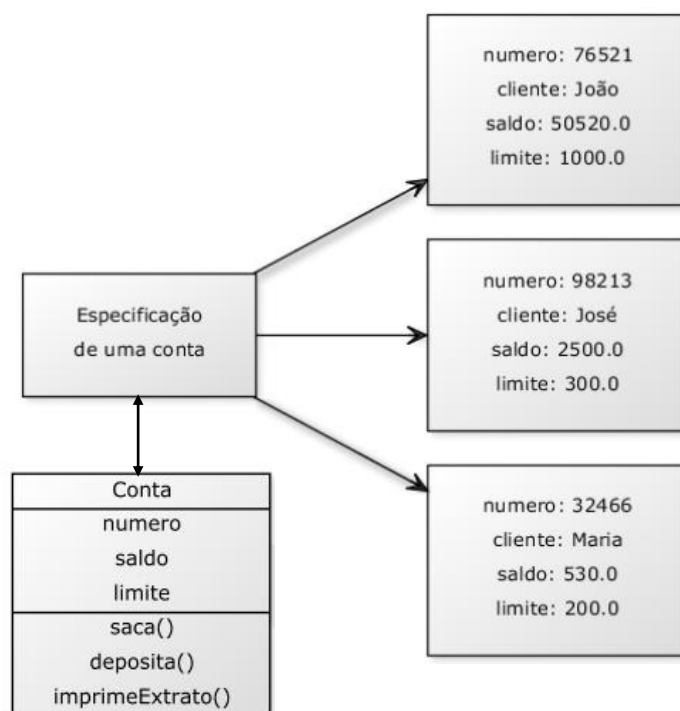


Figura 3. Criando objetos a partir da definição de uma classe.

Observe que na Figura 3 criamos mais um atributo para a classe conta, para fazer referência ao **nome do cliente** (atributo cliente). Cada atributo possui um **tipo** associado, da mesma forma que uma variável em qualquer Linguagem de Programação. Além disso, podemos definir métodos que possuem parâmetros, ou seja, valores que serão passados externamente para a classe. Um método pode também possuir um valor de retorno, que representa um valor de um dado tipo que é retornado quando da chamada do referido método. A Figura 4 ilustra outro exemplo de classe Conta em UML contendo os elementos acima citados.

Na Figura 4 observamos a classe Conta com apenas 2 atributos (número do tipo **int** e **saldo** do tipo **double**) e alguns métodos. O método **getNumero()**, por exemplo, retorna um valor inteiro correspondente ao número da conta; o método **depositar(valor:double)** não retorna nenhum valor, entretanto recebe um parâmetro correspondente ao valor **double** que atualizará o atributo saldo de algum objeto da classe conta; o método **sacar(valor: double)** recebe um parâmetro correspondente ao valor **double** que atualizará o atributo saldo de algum objeto da classe conta e retorna um valor lógico (**boolean**) que indica se o saque foi possível ou não.

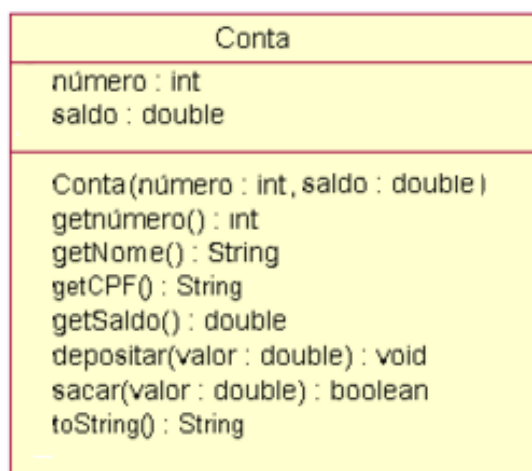


Figura 4. Representação UML mais completa da classe Conta, atributos e operações.

É importante observar, ainda na Figura 4, um método especial, chamado **Conta (numero: int, saldo: double)**, o chamado método construtor; um construtor é um

método executado **no momento da criação do objeto da classe**. Dessa forma, é possível definirmos construtores com parâmetros que receberão valores diferentes para a criação de objetos diferentes (embora possam existir dois objetos diferentes com o mesmo conjunto de atributos). No exemplo, para criar objetos da classe Conta chamamos o método construtor, passando o número da conta que desejamos criar e o correspondente saldo.

Objetos são criados no método construtor: Tal operação é definida na classe, assim como as demais operações, com a ressalva de que toda classe deve ter definida uma operação construtor obrigatoriamente, por padrão. Na operação construtor especificamos um método com ações que são executadas assim que o objeto for criado.

Passos para manipular classes e objetos:

- Definir atributos e operações da classe
- Criar objetos por meio do método construtor
- Manipular os objetos criados por meio da chamada das demais operações. Cada operação altera o **estado** dos atributos da classe, representando o **comportamento** do objeto.

Objetos são criados no método construtor: Tal operação é definida na classe, assim como as demais operações, com a ressalva de que toda classe deve ter definida uma operação construtor obrigatoriamente, por padrão. Na operação construtor especificamos um método com ações que são executadas assim que o objeto for criado.

Exemplo 1: Modelar a Classe **Carro** e ilustrar objetos dessa classe, identificando a utilização de operações para alterar as características dos objetos da classe.

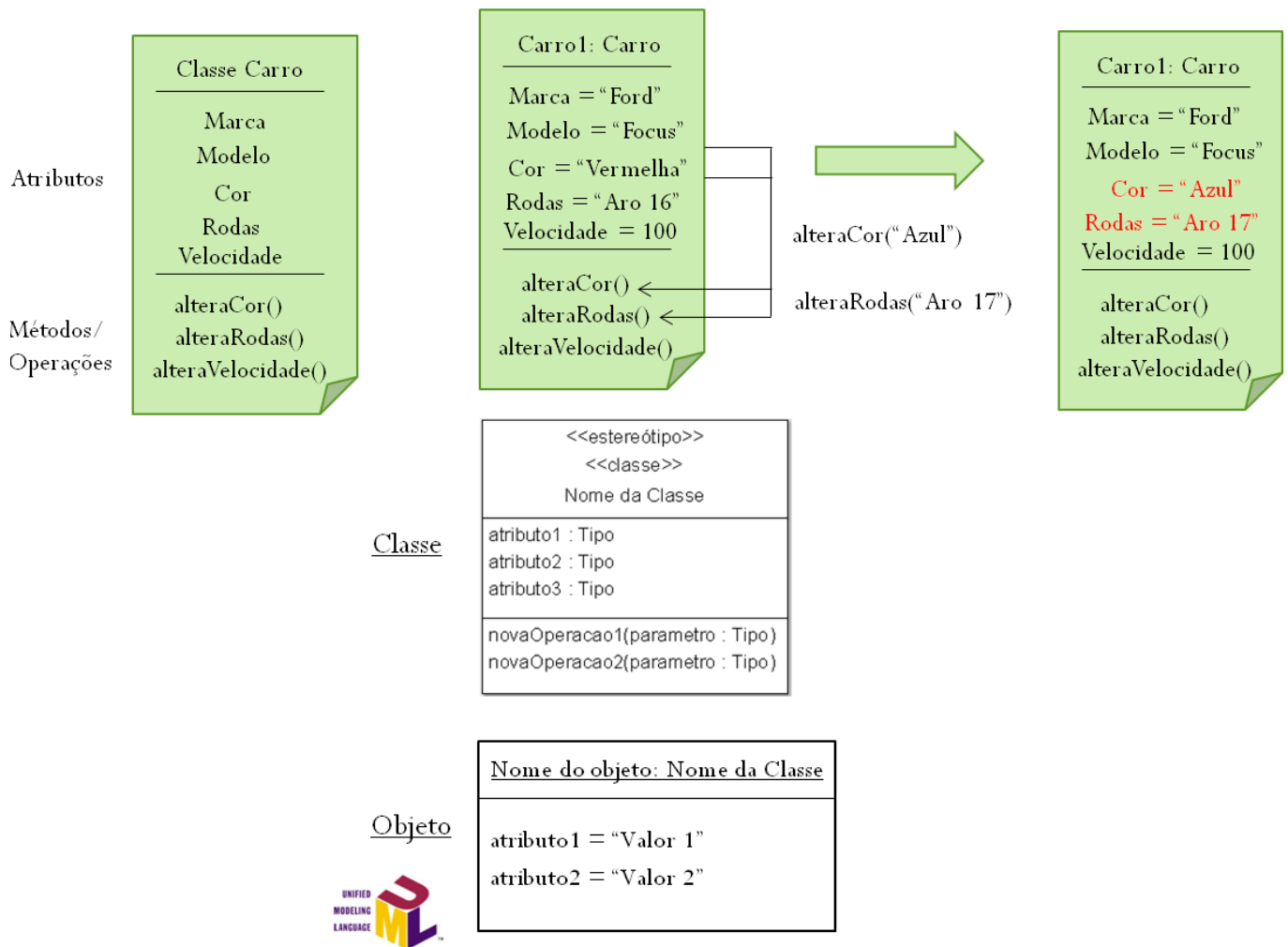


Figura 5. Representação de classes e objetos para a entidade Carro.

No Exemplo 1, a operação construtor é responsável por criar o objeto da classe carro, chamado carro1, com os atributos ilustrados. Por meio da chamada das operações, os demais métodos alteram o estado de alguns dos atributos do objeto. Na Figura 5 temos uma classe Carro e um objeto (chamado **Carro1**). Tal objeto tem seus atributos definidos no momento da chamada da operação construtor e posteriormente tem alguns de seus atributos modificados pela chamada das operações **alteraCor()** e **alteraRodas()**.

Exemplo 2: Modelar a classe Empregado, e as operações que podem ser realizadas sobre seus objetos. Criar objetos da classe Empregado e verificar a mudança das características de tais objetos por meio da chamada das operações da classe.

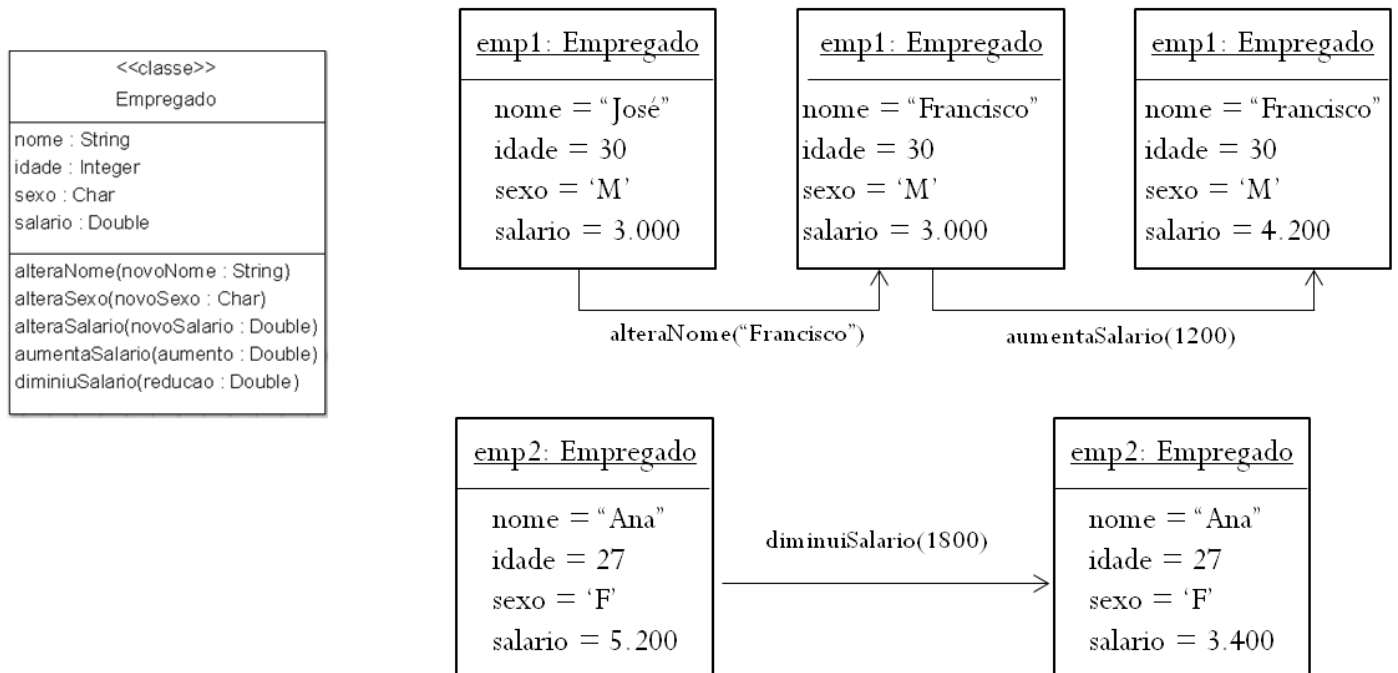


Figura 6. Classes e objetos para a entidade Empregado.

No Exemplo 2 vemos a definição da classe **Empregado**, com seus atributos e operações. Por meio da operação **construtor**, podemos observar a definição de dois objetos: **emp1** e **emp2**, com características diferentes. Em **emp1** são executadas duas operações, para alterar o nome e o salário do objeto, respectivamente, passando os parâmetros convenientes (o nome é alterado de Francisco Jose para Francisco e o salário é alterado de 3000 para 4200, +1200). A mesma lógica é aplicada ao objeto **emp2**.

0.4 Criando Classes e Objetos em Java

O conceito de classe apresentado anteriormente é genérico e pode ser aplicado em diversas linguagens de programação. Mostraremos como a classe Conta poderia ser escrita utilizando a linguagem Java. Inicialmente, discutiremos apenas sobre os atributos. Os métodos serão abordados posteriormente.

```

class Conta {
    int numero;
    String dono;
    double saldo;
    double limite;
}
  
```

```
// ..  
}
```

A classe Java Conta é declarada utilizando a palavra reservada **class**. No corpo dessa classe, são declaradas três variáveis que são os atributos que os objetos possuirão. Como a linguagem Java é estaticamente tipada, os tipos dos atributos são definidos no código. Os atributos saldo e limite são do tipo double, que permite armazenar números com casas decimais, e o atributo numero é do tipo int, que permite armazenar números inteiros.

A classe deve ser salva em diretório do Sistema Operacional com a extensão **.java** e com o arquivo com o **mesmo nome** da classe criada (no caso, **Conta.java**). Após definir a classe Conta, podemos criar objetos a partir dela. Esses objetos devem ser alocados na memória RAM do computador. Felizmente, todo o processo de alocação do objeto na memória é gerenciado pela máquina virtual. O gerenciamento da memória é um dos recursos mais importantes oferecidos pela máquina virtual.

Do ponto de vista da aplicação, basta utilizar um comando especial para criar objetos e a máquina virtual se encarrega do resto. O comando para criar objetos é o **new**.

```
class TestaConta {  
    public static void main(String[] args) {  
        // Comentário de Linha Criando um Objeto  
        //da Classe Conta  
        new Conta();  
        /**  
         * Comentário de Bloco...  
         **/  
    }  
}
```

Note que no exemplo acima temos uma seção de **comentários**. Em Java, comentários podem ser expressos em **linha**, após a sequência **//**, ou **em** bloco, delimitado pelos símbolos **/*** e ***/**. A linha com o comando **new** poderia ser repetida cada vez que desejássemos criar (instanciar) um objeto da classe Conta. A classe TestaConta (salva como **TestaConta.java**, no mesmo diretório da classe

Conta.java) serve apenas para colocarmos o método main, que é o ponto de partida da aplicação.

```
class TestaConta {  
    public static void main(String[] args) {  
        // Criando três Objetos da Classe Conta  
        new Conta();  
        new Conta();  
        new Conta();  
    }  
}
```

0.5 Referências

Todo objeto possui uma referência. A referência de um objeto é a única maneira de acessar os seus atributos e métodos. Dessa forma, devemos guardar as referências dos objetos que desejamos utilizar.

Ao utilizar o comando **new**, um objeto é alocado em algum lugar da memória. Para que possamos acessar esse objeto, precisamos de sua referência. O comando **new** devolve a referência do objeto que foi criado. Para guardar as referências devolvidas pelo comando **new**, devemos utilizar variáveis não primitivas (variáveis de tipos como inteiro, real, lógico, etc).

```
Conta referencia = new Conta ();
```

No código Java acima, a variável **referencia** receberá a referência do objeto criado pelo comando new. Essa variável é do tipo Conta. Isso significa que ela só pode armazenar referências de objetos do tipo Conta.

0.6 Manipulando Atributos

Podemos alterar ou acessar os valores guardados nos atributos de um objeto se tivermos a referência a esse objeto. Os atributos são acessados pelo nome. No caso específico da linguagem Java, a sintaxe para acessar um atributo utiliza o operador "." (Ponto).

```
Conta referencia = new Conta ();
```

```
referencia.saldo = 1000.0;
referencia.limite = 500.0;
referencia.numero = 1;
System.out.println(referencia.saldo);
System.out.println(referencia.limite);
System.out.println(referencia.numero);
```

No código acima, o atributo saldo recebe o valor 1000.0. O atributo limite recebe o valor 500 e o numero recebe o valor 1. Depois, os valores são impressos na tela através do comando **System.out.println** (Saída Padrão do Java que mostra na tela valores no formato de caracteres, ou objetos da classe **String**).

0.7 Valores Padrão

Poderíamos instanciar um objeto e utilizar seus atributos sem inicializá-los explicitamente, pois os atributos são inicializados com valores padrão. Os atributos de tipos **numéricos** são inicializados com **0**, os atributos do tipo **boolean** são inicializados com **false** e os demais atributos com **null** (referência vazia).

```
class Conta {
    double limite;
}
```

```
class TestaConta {
    public static void main( String [] args ) {
        Conta conta = new Conta ();
        // imprime 0(zero)
        System.out.println(conta.limite);
    }
}
```

A inicialização dos atributos com os valores padrão ocorre na instanciação, ou seja, quando o comando **new** é utilizado. Dessa forma, todo objeto “nasce” com os valores padrão. Em alguns casos, é necessário trocar esses valores. Para trocar o

valor padrão de um atributo, devemos inicializá-lo na declaração. Por exemplo, suponha que o limite padrão das contas de um banco seja R\$ 500. Nesse caso, seria interessante definir esse valor como padrão para o atributo limite.

```
class Conta {  
    double limite = 500;  
}
```

```
class TestaConta {  
    public static void main ( String [] args ) {  
        Conta conta = new Conta ();  
        // imprime 500  
        System.out.println(conta.limite);  
    }  
}
```

0.8 Métodos

Em um banco é possível realizar diversas operações em uma conta: depósito, saque, transferência, consultas e etc. Essas operações podem modificar ou apenas acessar os valores dos atributos dos objetos que representam as contas. Essas operações são realizadas em **métodos** definidos na própria classe Conta. Por exemplo, para realizar a operação de depósito, podemos acrescentar o seguinte método na classe Conta

```
void deposita(double valor) {  
    // Implementação do Método  
    saldo = saldo + valor;  
}
```

Podemos dividir o método em quatro partes, ilustradas na Figura 7:

- Nome: É utilizado para chamar o método. Na linguagem Java, é uma boa prática definir os nomes dos métodos utilizando a convenção “ Camel Case” com a primeira letra minúscula.

- Lista de Parâmetros: Define os valores que o método deve receber. Métodos que não devem receber nenhum valor possuem a lista de parâmetros vazia.
- Corpo: Define o que acontecerá quando o método for chamado
- Retorno: A resposta que será devolvida ao final do processamento do método. Quando um método não devolve nenhuma resposta, ele deve ser marcado com a palavra reservada **void**.

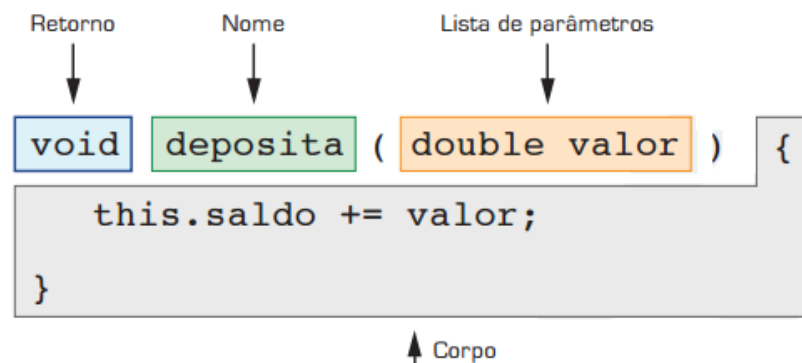


Figura 7. Partes que compõem um método.

Para realizar um depósito, devemos chamar o método **deposita()** através da referência do objeto que representa a conta que terá o dinheiro creditado.

```
// Referência de um objeto
Conta c = new Conta();
// Chamando o método deposita()
c.deposita(1000)
```

Normalmente, os métodos acessam ou alteram os valores armazenados nos atributos dos objetos. Por exemplo, na execução do método **deposita()**, é necessário alterar o valor do atributo **saldo** do objeto que foi escolhido para realizar a operação. Dentro de um método, para acessar os atributos do objeto que está processando o método, devemos utilizar a palavra reservada **this**. Dessa forma, o método **deposita()** pode ser modificado para:

```
void deposita(double valor) {
    // Implementação do Método
    this.saldo += valor;
    // += é o operador de soma, que incrementa
    // automaticamente o valor da variável soma,
    // incluindo a variável valor
}
```

```
}
```

O método **deposita()** não possui nenhum retorno lógico. Por isso, foi marcado com **void**. Mas, para outros métodos, pode ser necessário definir um tipo de retorno específico. Considere, por exemplo, um método para realizar a operação que consulta o saldo disponível das contas. Suponha também que o saldo disponível é igual a soma do saldo e do limite. Então, esse método deve somar os atributos saldo e limite e devolver o resultado. Por outro lado, esse método não deve receber nenhum valor, pois todas as informações necessárias para realizar a operação estão nos atributos dos objetos que representam as contas. Assim temos:

```
double ConsultaSaldoDisponivel() {  
    // Implementação do Método  
    return this.saldo + this.limite;  
}
```

Ao chamar o método **consultaSaldoDisponivel()** a resposta pode ser armazenada em uma variável do tipo **double**:

```
class TestaConta () {  
  
    public TestaConta main(String[] args){  
        Conta c = new Conta();  
        c.deposita(1000);  
        // Armazenando o retorno em uma variável  
        // double  
        double sDisp = c.consultaSaldoDisponivel();  
        System.out.println("Saldo Disponível: " + sDisp);  
    }  
}
```

0.9 Sobrecarga (overloading)

A sobrecarga ocorre quando temos mais de um método com a mesma assinatura, mas com um conjunto de atributos diferentes. A assinatura (nome) de métodos pode ser a mesma, entretanto o conjunto de parâmetros é diferente. Podemos sobrecarregar tanto métodos comuns da classe como também o **método**

construtor. Por exemplo: considere a classe empregado, com os seguintes atributos especificados no código java a seguir (Empregado.java):

```
/**
 * Classe Empregado,
 * que representa atributos e operações de um empregado.
 * */
class Empregado {

    String nome;
    int idade;
    double salario;
    char sexo;

    Empregado(String nome, int idade, double salario, char sexo){
        this.nome = nome;
        this.idade = idade;
        this.salario = salario;
        this.sexo = sexo;
    }

    // Criando uma operação para aumentar o salário do Empregado
    void aumentaSalario(double aumento){
        this.salario += aumento;
    }

    // Criando uma operação para aumentar o salário em 10%
    void aumentaSalario(){
        this.salario = this.salario + 0.10*this.salario;
    }

    // Uma outra operação aumentaSalario com o aumento absoluto e
    relativo
    void aumentaSalario(double aumento, double taxa){
        this.salario = this.salario + taxa*this.salario + aumento;
    }

    void mostraInformacoes(){
        System.out.println("Informações do Empregado: ");
        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade);
        System.out.println("Salário: " + salario);
        System.out.println("Sexo: " + sexo);
    }
}
```

Note que na classe Empregado definimos quatro atributos: nome, idade, salario e sexo. O método **aumentaSalario** está sobrecarregado, ou seja, ele pode ser executado de três formas e, dependendo dos parâmetros que forem passados, uma de suas versões será chamada. Na listagem seguinte temos a utilização da classe Empregado.java (classe TestaEmpregado.java):

```

class TestaEmpregado {

    public static void main(String args[]){

        Empregado emp1 = new Empregado("Maria", 20, 1500, 'F');
        Empregado emp2 = new Empregado("João", 30, 2000, 'F');

        emp1.aumentaSalario();
        emp1.aumentaSalario(1500);

        emp2.aumentaSalario(100, 0.25);

        emp1.mostraInformacoes();
        emp2.mostraInformacoes();
    }
}

```

Na listagem acima verificamos a declaração de dois objetos da classe **Empregado**: **emp1** e **emp2**. Os dois diferem em seus atributos, de acordo com os valores que foram passados para os construtores de cada um. Nos dois casos, a chamada da palavra **new**, como visto, automaticamente chamará o construtor da classe **Empregado**, e inicializará os valores de cada objeto com os argumentos passados nos parâmetros. No caso de **emp1**, a execução do **aumentaSalario()** atualizará o valor da variável **salario**, em 10%, de 1500 para $1500 + 150 = 1650$ e, em seguida, a execução da operação **aumentaSalario(1500)**, atualizará o valor da variável para $1650 + 1500 = 3150$. Para **emp2**, a chamada de **aumentaSalario(100, 0.25)** atualizará o valor da variável para $2000 + 100 + 0.25 * 2000 = 2600$. Os valores mostrados na saída serão, portanto:

```

Informações do Empregado:
Nome: Maria
Idade: 20
Salário: 3150.0
Sexo: F
Informações do Empregado:
Nome: João
Idade: 30
Salário: 2600.0
Sexo: F

```

Podemos ainda ter **sobrecarga de construtor**, ocorrendo a possibilidade de definirmos um ou mais construtores para a nossa classe, mudando apenas a

quantidade ou a ordem ou o tipo dos parâmetros passados. Por exemplo, considere a classe Conta.java a seguir:

```
class Conta {

    int numero;
    double saldo;
    String cliente;

    // CONSTRUTOR 1: Definindo o construtor padrão de conta,
    // com todos os valores nulos
    Conta(){
        numero = 0;
        saldo = 0;
        cliente = "";
    }

    // CONSTRUTOR 2: Definido um construtor da classe Conta apenas
    // com o numero da conta e os demais valores nulos
    Conta(int numero){
        saldo = 0;
        cliente = "";
        this.numero = numero;
    }

    // CONSTRUTOR 3: Definindo um construtor da classe Conta com o
    // numero e saldo, e com o nome vazio
    Conta(int numero, double saldo){
        this.numero = numero;
        this.saldo = saldo;
        cliente = "";
    }

    // CONSTRUTOR 4: Definido um construtor da Classe Conta apenas
    // com o nome do cliente, e os demais valores vazios...
    Conta(String cliente){
        this.cliente = cliente;
        saldo = 0;
        numero = 0;
    }

    String retornaNomeCliente(){
        return cliente;
    }

    double retornaSaldo(){
        return saldo;
    }

    int retornaNumero(){
        return numero;
    }

}
```

Na classe Conta acima, temos a definição de **quarto** construtores, todos podendo criar objetos da classe conta. Entretanto, cada referência criada para a classe Conta,

terá os atributos definidos para o objeto, **dependendo do construtor que for chamado**. Por exemplo, se for chamado, para criar um objeto, o construtor **Conta(1001)**, será entendido que deverá ser chamado o **construtor 2**, para criar uma conta apenas com o atributo **numero** 'preenchido'; entretanto, se for chamado o construtor **Conta(Maria)**, será entendido que deverá ser executado o **construtor 4**, para definir objeto apenas com a variável **cliente** 'preenchida'. A classe TesteConta a seguir ilustra a idéia:

```
class TesteConta {  
  
    public static void main(String[] args) {  
  
        // Criando um objeto com todos os atributos nulos  
        Conta c1 = new Conta();  
        // Criando uma conta apenas com número  
        Conta c2 = new Conta(111);  
        // Criando uma conta apenas com número e saldo  
        Conta c3 = new Conta(111, 2550.92);  
        // Criando uma conta apenas com o nome do cliente  
        Conta c4 = new Conta("Maria");  
  
        // Modificando os atributos das contas..  
        c1.cliente = "Jose";  
        c1.numero = 999;  
        c1.saldo = 3000;  
  
        c2.numero = 200;  
        c2.cliente = "Pedro";  
  
        c3.cliente = "Ana";  
  
        c4.numero = 992;  
        c4.saldo = 100;  
  
        // Mostrando os atributos de c4:  
        System.out.println("Conta c4: ");  
        System.out.println("Numero " + c4.retornaNumero());  
        System.out.println("Cliente: " + c4.retornaNomeCliente());  
        System.out.println("Saldo: " + c4.retornaSaldo());  
    }  
}
```

Na classe TestaConta acima temos a criação de 4 objetos, cada um dos quais com um construtor diferente. Embora os objetos c2 e c4 sejam criados com a chamada Conta(valor), para cada um deles é chamado um construtor diferente, pois no caso de c2 temos um valor **int** passado no parâmetro e no caso de c4 temos um valor **String**. Em seguida à criação dos objetos, alteramos diretamente os atributos de

cada um deles, de forma que todos os valores estejam 'preenchidos' (não tenhamos nenhum valor nulo ou vazio). Por fim, temos a chamada aos métodos **retornaNumero()**, **retornaNomeCliente()** e **retornaSaldo()**, que mostram os valores correspondentes ao objeto c4.

Podemos ainda ter um método sobrecarregado chamando outro método sobrecarregado ou construtor. Na classe **Conta** poderíamos ter definido um quinto construtor da seguinte forma:

```
Conta(String nomeCliente, int numero, double saldo){  
    this(numero, saldo); // Aqui chamamos o construtor c3!  
    this.cliente = nomeCliente;  
}
```

Esse construtor, por meio da instrução **this(numero, saldo);** chama o construtor **c3** da classe **Conta** para inicializar os campos **numero** e **saldo** e inicializa diretamente o campo **cliente**. Não podemos, por meio de um construtor criado, chamar mais de um construtor, por exemplo, a definição do quinto construtor conforme abaixo é **inválida**:

```
Conta(String nomeCliente, int numero, double saldo){  
    this(numero, saldo);  
    this(nomeCliente); // Linha inválida! Erro!  
}
```

Exercícios da Unidade

1. Qual a importância da Programação Orientada a Objetos? Quais as diferenças entre **classe** e **objeto**?
2. Implemente uma classe para definir os objetos que representarão os clientes de um banco. Essa classe deve declarar dois atributos: um para os nomes e outro para os códigos dos clientes. Crie o construtor da classe **Cliente**, que deve inicializar os campos da mesma. Crie uma classe (com o método **main**) para testar a criação de um objeto da classe **Cliente** e a alteração de seus atributos.
3. Implemente a classe **Livro** com os atributos: **titulo (String)**, **autor (String)**, **editora (String)** e **ano (int)**. Defina um construtor da classe, para inicializar todos os seus atributos. Crie a classe **TestaLivro**, com o método **main**, dentro do qual você deverá criar dois objetos da classe **Livro** (com criação das

referências com a palavra **new**). Acesse todos os atributos dos dois objetos criados e mostre seus valores.

4. Crie uma classe Produto, com dois atributos: nome (String) e preco (double). Crie métodos na classe Produto: **void mostraNome()**, para mostrar o nome (do tipo **void**, que chamará o System.out.println para mostrar o valor do atributo nome) e outro **double retornaPreço()**, para retornar o valor do preço do produto. Crie ainda um terceiro método void **aumentaPreco(double porcentagem)** para aumentar o valor da variável preco do produto de uma porcentagem passada no parametro e o ultimo método **diminuiPreco(double porcentagem)** para diminuir o preco do produto de um valor passado no parametro. Teste a classe Produto, na classe TesteProduto, que terá o método **main**, dentro do qual deverão ser criados dois objetos da classe produto p1(" arroz" com preço de 350) e p2(" açúcar" ,com preço de 400) Efetuar ainda na classe TesteProduto um aumento de 35% no preço do arroz e redução de 40% no preço do açúcar.
5. Crie a classe retangulo com os atributos **base** e **altura**, ambos **double**. Crie dois construtores: um para inicializar os atributos como **zero** e outro para receber os valores dos atributos. Crie um método **void calculaArea()** para calcular e mostrar a área do retângulo. Teste a classe criada, receba os atributos e chame o método **calculaArea()**.

1 CRIANDO CLASSES E OBJETOS COM JAVA

Neste capítulo continuaremos com a definição e criação de classes em Java. Especificamente serão abordados mais dois conceitos fundamentais do paradigma OO: **Encapsulamento** e **Campos/Métodos** abstratos. Aqui também aprenderemos a criar e modificar projetos OO com Java no Eclipse.

1.1 Requisitos para Instalação do Ambiente de Desenvolvimento

Antes de iniciar o desenvolvimento em uma ferramenta de programação (no caso, o Eclipse IDE) é fundamental a instalação do Ambiente de Desenvolvimento Java. Na página <http://www.oracle.com/technetwork/pt/java/javase/downloads/index.html> está disponível a versão mais recente do JSDK (Java Standard Development Kit), o suporte fundamental para o desenvolvimento de aplicações Java. A versão utilizada

por esta apostila é a Java Platform (JDK) 8u65 / 8u66 (para plataforma Windows, Sistema Operacional de 64bits). Na página estão disponíveis versões para outras arquiteturas e Sistemas Operacionais. O JSDK contém dois componentes principais: JDK, o ambiente de desenvolvimento, que contém bibliotecas e recursos para a compilação e interpretação do código-fonte em Java. Com tais ferramentas é possível transformar um arquivo Java de classe em um executável, ou arquivo **.class**. A segunda ferramenta é a JRE (Java Runtime Environment), que permite a **execução** do arquivo **.class**. A instalação é relativamente simples, e a utilização de IDEs (como NetBeans IDE ou Eclipse IDE) facilita bastante a tarefa de configuração e execução das aplicações.

1.2 Breve Introdução ao Ambiente Eclipse IDE

Após a instalação do JSDK é necessário o download e instalação do Eclipse IDE. O Eclipse (<https://eclipse.org/>) é um ambiente Integrado de Desenvolvimento, que permite a rápida criação de aplicações OO escritas em Java, além de suporte para alguns ambientes de programação como C++/PHP/JavaScript e plataformas como ApacheTomCat, suporte a Hibernate, Suporte a Programação Web com Java, etc (mais detalhes na página de instalação em <https://eclipse.org/downloads/>).

Após o download e instalação da versão correta da IDE (recomenda-se a versão **Eclipse IDE for Java Developers**, disponível em: <http://www.eclipse.org/downloads/packages/eclipse-ide-java-developers/mars1>, observar o Sistema Operacional e a plataforma x86 ou x64), a primeira interface na execução da ferramenta mostra uma janela para configurar o **Workspace**, ou área de projetos: o local onde os projetos desenvolvidos serão armazenados. Selecione um endereço e avance para a janela seguinte. A interface inicial da ferramenta é mostrada na Figura 8:

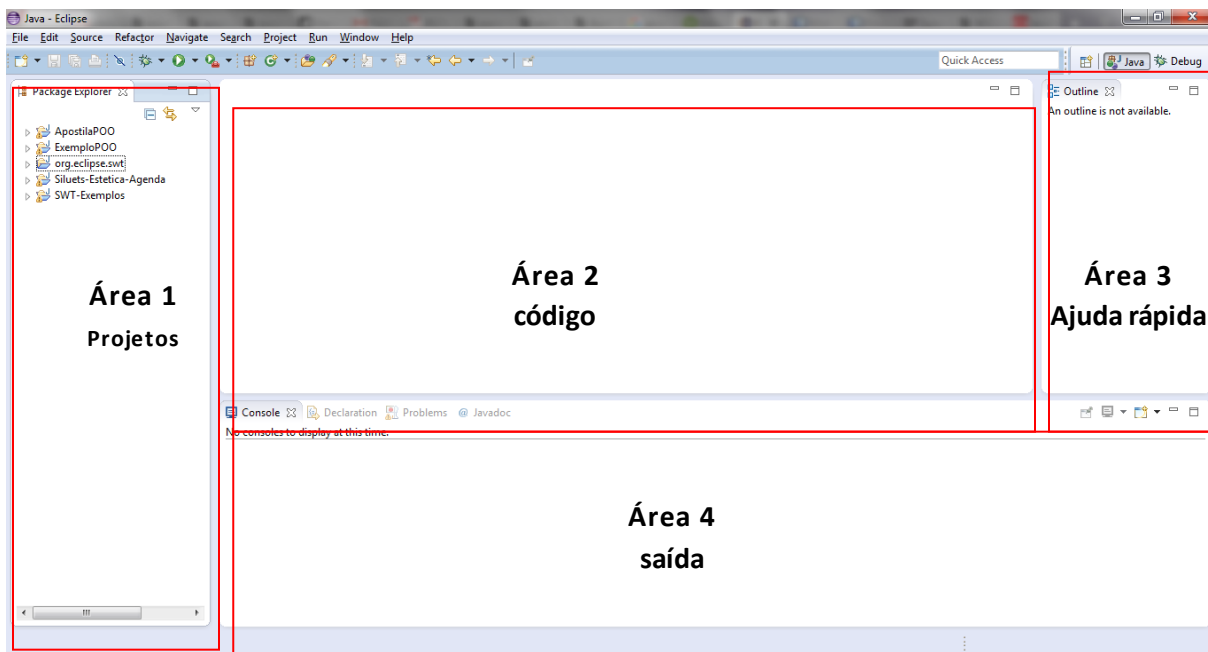


Figura 8. Tela Inicial do Eclipse IDE.

Na Figura 8, a área 1 (Projetos) representa a área de projetos, onde todos os projetos de aplicações serão visualizados em árvore; é possível visualizar individualmente cada elemento de cada projeto, na medida em que se navega na estrutura em árvore; na área 2 (Código) é possível visualizar o código-fonte da classe em questão no projeto; na área 3 (Ajuda rápida) é possível obter ajuda online sobre algum item da ferramenta, tem-se o código-fonte do projeto, onde se modifica o código das classes. Por fim, na área 4 (saída ou console) tem-se a saída padrão, onde são mostradas os resultados impressos no código (principalmente as saídas por meio do comando **System.out.print**). Mensagens de possíveis erros também são mostradas nessa área. A aparência do Eclipse pode no entanto ser customizada com diferentes visões sobre os elementos do projeto (nas opções Window>>Show View e Window>>Customize Perspective).

Para criar um novo projeto seleciona-se File>>New>>Java Project. A janela da Figura 9 aparecerá:

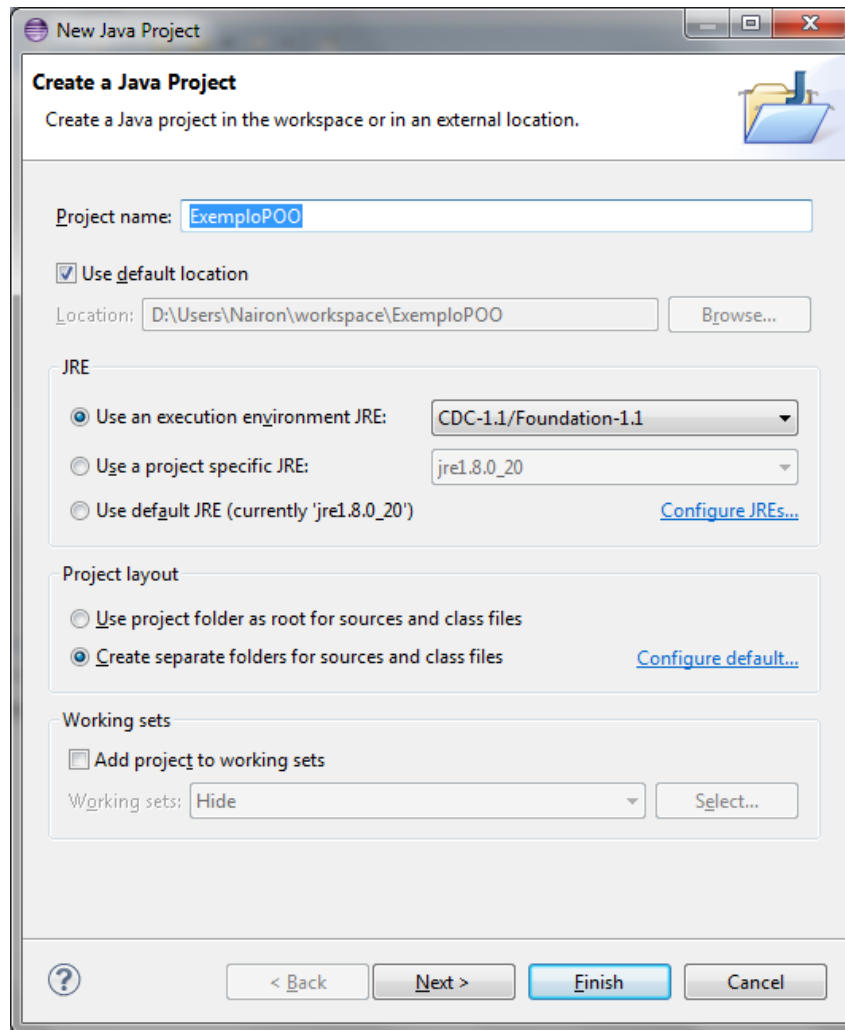


Figura 9. Novo Projeto no Eclipse.

Na janela da Figura 9 é possível configurar o nome do projeto e algumas outras características como a versão do ambiente JRE (na figura, versão 1.8.0_20). Ao clicar em Next janela da Figura 10 aparecerá:

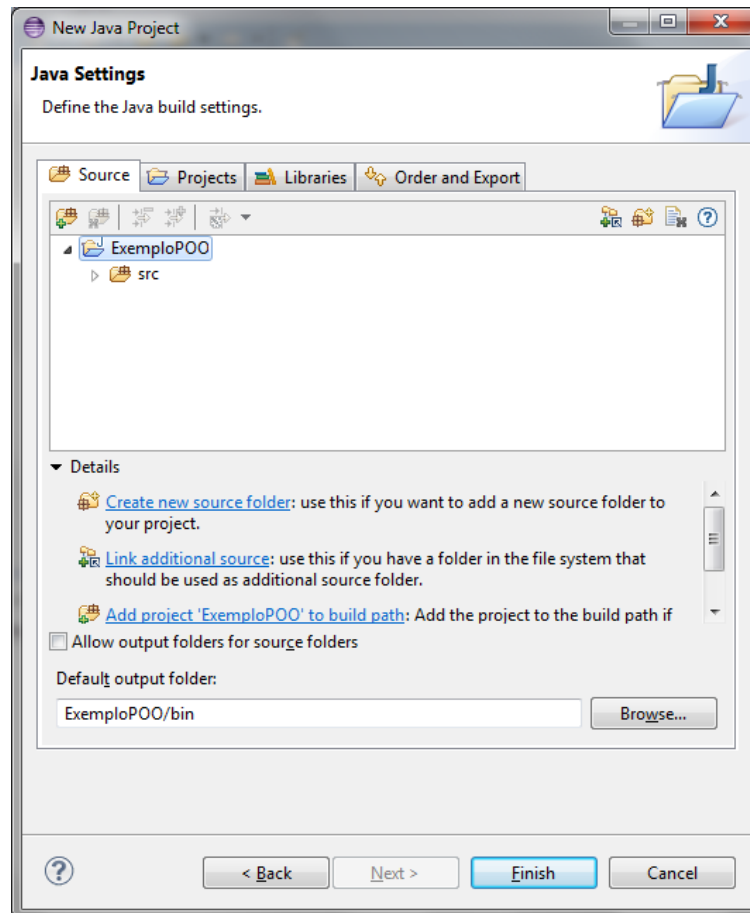


Figura 10. Configuração do Projeto no Eclipse.

Após clicar em "Finish" um novo Projeto (no caso, ExemploPOO) será criado. A estrutura do projeto, na Área 1, será organizada conforme a Figura 11:

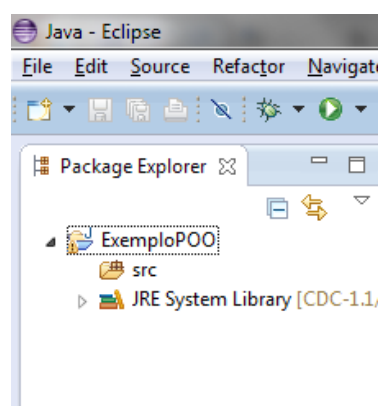


Figura 11. Estrutura do Projeto Criado no Eclipse IDE.

No diretório <src>, devemos organizar o código fonte das classes a serem utilizadas no Projeto. Classes são organizadas em **pacotes**. Um pacote em Java (**package**) é um agrupamento de classes com funções relacionadas. Por exemplo, para um domínio de um sistema bancário, podemos ter um pacote para armazenar as classes de Conta e outro pacote para armazenar as classes Cliente; para um sistema acadêmico, podemos ter um pacote para armazenar as classes Disciplina e Turma e outro pacote para armazenar as classes Professor, Diretor e Aluno. Para criar um novo pacote selecionamos o item <src>, Botão Direito > New> Package. Vamos criar os seguintes pacotes: funcionários, produtos e clientes. A estrutura do projeto será então a da Figura 12:

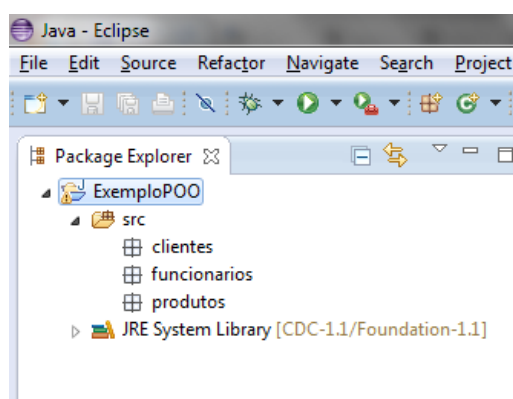


Figura 12. Organização do Projeto em Pacotes no Eclipse.

Por fim, em cada um dos pacotes, podemos agrupar as classes para trabalho. No pacote **clientes** criamos a classe **Cliente**; no pacote **funcionários** criamos a classe **Empregado** e no pacote **produtos** criamos a classe **Livro**. Para criar uma nova classe selecionamos o pacote em questão e Botão Direito >> New >> Class. Preenchemos o nome da classe no campo correspondente e, após a criação das classes em cada pacote a seguinte estrutura do projeto é definida na Figura 13.

Automaticamente o Eclipse abre uma janela na área de trabalho como “esqueleto”

(um template padrão) do código da classe gerado. Vale observar, para cada classe, a cláusula **package** <nome do pacote>; que indica que a classe em questão está associada àquele pacote. Uma vez removida do pacote o novo pacote deve ter seu nome atualizado na cláusula **package** novamente. Por exemplo, o código inicial da classe Livro no pacote produto é expresso na Listagem da tabela em seguida:

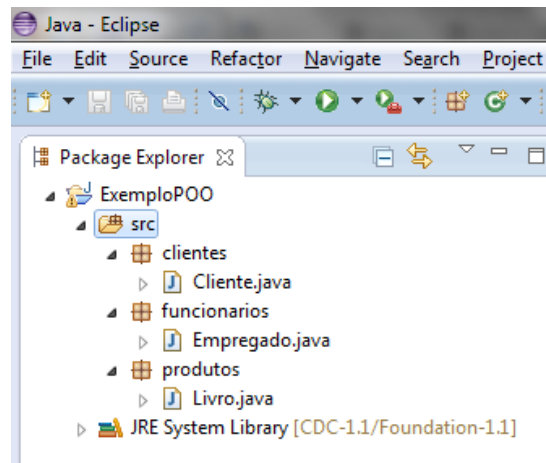


Figura 13. Estrutura do Projeto Eclipse com pacotes e classes.

```
package produtos;

public class Livro {

}
```

Outra observação é o uso da cláusula **public** que indica a **visibilidade da classe**, a ser discutida na próxima seção. Para testar as classes do projeto devemos definir uma classe com o método principal (**main**) dentro da qual criaremos os objetos. A classe principal pode ser criada em qualquer um dos três pacotes (poderíamos, por exemplo, criar uma classe principal para cada um dos três pacotes). No entanto, vamos definir um último pacote (**teste**) dentro do qual criaremos a classe **Principal.java**, dentro da qual criaremos objetos das três classes. A estrutura do projeto deverá ser atualizada conforme a Figura 14:

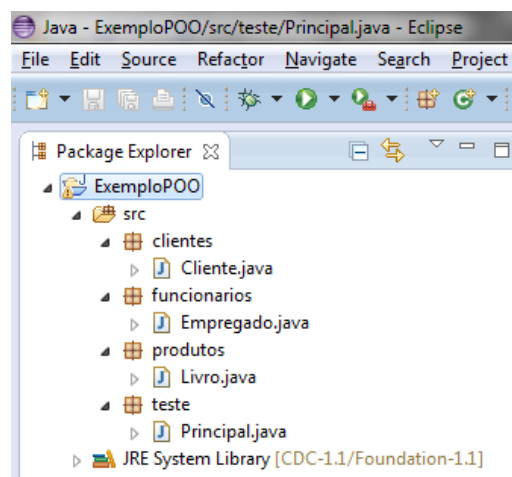


Figura 14. Estrutura final do Projeto Eclipse com pacotes e classes.

Dentro da classe Principal criaremos três objetos, um para cada classe. Para isso é necessário **importar** cada uma das classes utilizadas pela classe principal. A **importação de classes** é necessária quando uma classe está em pacote diferente de outra classe e o ambiente do interpretador Java não consegue encontrá-la diretamente. Se as classes Livro, Empregado e Cliente não estão no mesmo pacote da classe Principal então elas devem ser importadas por meio do uso da cláusula **import**. O código da classe Principal.java é portanto:

```
package teste;

import produtos.Livro;
import clientes.Cliente;
import funcionarios.Empregado;

/**
 * Classe para testar as classe Empregado, Cliente e Livro
 * */
public class Principal {

    public static void main(String[] args){

        Cliente    c1 = new Cliente();
        Empregado  e1 = new Empregado();
        Livro      l1 = new Livro();

    }

}
```

Dentro da classe Principal criaremos três objetos, um para cada classe. Para isso é necessário **importar** cada uma das classes utilizadas pela classe principal. A **importação de classes** é necessária quando uma classe está em pacote diferente de outra classe e o ambiente do interpretador Java não consegue encontrá-la diretamente. Se as classes Livro, Empregado e Cliente não estão no mesmo pacote da classe Principal então elas devem ser importadas por meio do uso da cláusula **import**. O código da classe Principal.java é portanto:

1.3 Visibilidade e Encapsulamento

Atributos e métodos de classes podem ser acessados diretamente por outras classes. Por exemplo, considere a seguinte classe Produto, com atributos nome e preço:

```

class Produto {

    String nome;
    double preco;

    Produto(String nome, double preco){
        this.nome = nome;
        this.preco = preco;
    }

}

```

Na classe principal podemos criar um objeto da classe Produto e acessar seus atributos da seguinte forma:

```

public static void main(String[] args){

    Produto p = new Produto("Arroz", 2.55);
    System.out.println("Nome: " + p.nome);
    System.out.println("Preco: " + p.preco);

    p.nome = "Feijao";
    p.preco = 4.55;

    System.out.println("Nome: " + p.nome);
    System.out.println("Preco: " + p.preco);

}

```

Entretanto, em muitas vezes ocorre a necessidade de não se permitir o acesso direto à atributos e operações por meio de outra classe. Por exemplo, seria mais interessante e conveniente, como uma boa prática de Programação Orientada a Objetos, não permitir que outra classe acessasse e modificasse diretamente os atributos de um objeto da classe Produto. Para garantir tal 'proteção' cada atributo e operação é especificado com uma **visibilidade**.

A **visibilidade** de atributos/métodos define se um atributo/método poderá ou não ser acessado de um objeto de outra classe. Existem basicamente três tipos de visibilidade, conforme na Figura 15:

Visibilidade	Descrição	Simbologia
público	o atributo/método pode ser acessado de dentro da classe ou de fora dela (representado pelo símbolo "+")	+
privado	o atributo/método pode ser acessado apenas pela própria classe	-
protegido	O atributo/operação pode ser acessado pela classe e por suas classes herdeiras	#

Figura 15. Tipos de visibilidade de atributos/operações em Java.

Em geral definem-se os atributos de uma classe como privados e as operações como públicas. Isso previne que classes externas acessem diretamente e modifiquem os atributos de outra classe. Atributos e operações privadas são definidas com o modificador **private**, antes da declaração do atributo. Já atributos e operações públicas são definidos como **public**. Por padrão, a cláusula **public** também é utilizada na definição de classes (**public class...**). Já atributos protegidos são definidos com a cláusula **protected**, que é utilizada quando da existência da relação de herança entre classes, a ser vista no capítulo 4.

Encapsular um atributo é proteger o seu acesso direto, expondo o acesso ao atributo apenas por meio da "publicação" de um método. Dessa forma, atributos encapsulados são modificados internamente, sem a necessidade de expor detalhes dessas modificações às classes externas. O **encapsulamento** determina como é realizada a operação do método público, mas a classe que solicitou o método não precisa dos detalhes de seu funcionamento. Considere o exemplo da Figura 17:

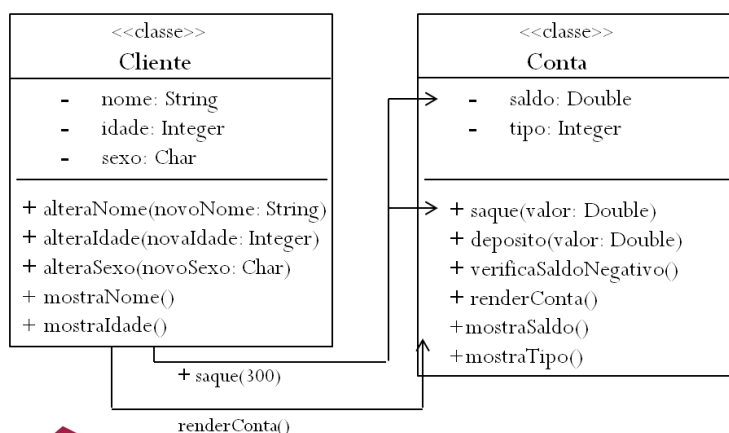


Figura 17. Exemplo de encapsulamento e visibilidade.

Visibilidade: objetos da classe Cliente não podem acessar diretamente atributos da classe Conta. O acesso a tais atributos é privado e permitido apenas por meio dos métodos públicos da classe Conta

Encapsulamento: ao solicitar os métodos **saque** e **renderConta** da classe Conta, um objeto da classe Cliente não precisa saber como esses métodos foram implementados, e de fato não sabe pois a implementação de tais

Ainda no exemplo da classe Produto, podemos definir o encapsulamento da seguinte forma:

- Os atributos da classe são definidos com a cláusula **private**: dessa forma, nenhuma class pode ter acesso direto aos atributos **nome** e **preco**. Entretanto, a classe principal não poderá modificar tais atributos
- Definir métodos **públicos** para **retornar** o valor de cada um dos atributos, bem como métodos para **alterar** o valor de tais atributos: dessa forma, quando qualquer classe externa for acessar um atributo da classe produto, este não será mais acessado diretamente (pois é privado), mas poderá ser acessado e modificado por meio de métodos públicos (um **void** para alterar o valor e outro para retornar o valor)

Assim, o código da classe produto, para estar de acordo com os padrões de visibilidade e encapsulamento deverá ser definido da seguinte forma:

```
package sample;

/**
 * A classe Produto é definida como pública, pois
 * ela por si só pode ser acessada por outras classes
 * de forma a ser possível a criação de objetos Produto.
 * */
public class Produto {

    /**
     * Os atributos da classe são privados, dessa forma
     * não podem ser modificados diretamente
     * */
    private String nome;
    private double preco;

    /**
     * Método construtor, que por padrão é público, para
     * permitir o acesso e criação de objetos da classe Produto
     */
}
```



```

    */
    public Produto(String nome, double preco){
        this.nome = nome;
        this.preco = preco;
    }

    /**
     * Método público para alterar o valor do preço
     */
    public void alteraPreco(double preco){

        // Proteção do atributo...
        // Verifica se o novo valor do Preço é nulo
        // Não pode haver produto com o preço nulo
        if(preco>=0)
            this.preco = preco;
    }

    /**
     * Método público para retornar o valor do preço
     */
    public double retornaPreco(){
        return this.preco;
    }

    /**
     * Método público para alterar o valor do nome
     */
    public void alteraNome(String nome){

        // Proteção do atributo...
        // Verifica se o novo valor do nome é vazio
        // Não pode haver produto com o nome vazio
        if(nome!="")
            this.nome = nome;
    }

    /**
     * Método público para retornar o nome do produto
     */
    public String retornaNome(){
        return this.nome;
    }
}

```

Da forma como foi definida, a classe do método principal executável (main) não terá mais acesso direto aos atributos da classe produto. Tais atributos estão **encapsulados** e o acesso aos mesmos só se dá por meio dos métodos definidos **alteraPreco()**, **retornaPreco()**, **alteraNome()**, **retornaNome()**.

A utilidade do encapsulamento pode ser vista na listagem acima. Por exemplo, anteriormente, a classe principal poderia arbitrariamente alterar o valor do preço do produto, inclusive inserindo um preço nulo ou negativo. Na nova versão da classe

Produto, o método `public void alteraPreco(double preco)` garante a proteção do atributo preço, uma vez que na sua implementação, o preço só poderá ser alterado se for maior ou igual a zero. Da mesma forma, `public void alteraNome(String nome)` garante que o atributo nome só poderá ser alterado se o novo nome for diferente de nulo (operador `!=`). A classe de teste deverá ser modificada para que possa ter o mesmo efeito, da seguinte forma:

```
public class Principal {

    public static void main(String[] args){

        Produto p = new Produto("Arroz", 2.55);
        System.out.println("Nome: " + p.nome);
        System.out.println("Preco: " + p.preco);

        p.alteraNome("Feijao");
        p.alteraPreco(4.55);
        p.alteraPreco(p.retornaPreco() + 2); // aumentará em 2.00
        p.alteraPreco(-10); // tal modificação não será realizada

        p.nome = "Milho"; // erro! Não pode ser acessado!
        p.preco = 2.22; // erro! Não pode ser acessado!

        System.out.println("Nome: " + p.retornaNome());
        System.out.println("Preco: " + p.retornaPreco());
    }

}
```

A ideia de encapsulamento é utilizada por padrão na maioria dos projetos de modelagem OO em Java. É uma recomendação extremamente importante, pois garante a proteção dos atributos da classe. Além de atributos, podemos também ter métodos privados: são métodos que só podem ser acessados **de dentro da classe**, não podendo ser chamados de outras classes. No exemplo da classe produto, podemos criar dois métodos privados internos, para verificar se o preço é inválido e para checar se o nome não é nulo. Os métodos ficariam da seguinte forma:

```
/**
 * Método privado para verificar se o preço do
 * produto é inválido. Esse método só tem sentido
 * se usado dentro da classe
 * */
private boolean precoValido(double preco){
    if(preco>=0)
        return true;
    else
```

```

        return false;
    }

    /**
     * Método privado para verificar se o nome do
     * produto não é nulo. Esse método só tem sentido
     * se usado dentro da classe
     */
    private boolean nomeNulo(String nome) {
        if (nome == "")
            return true;
        else
            return false;
    }

    /**
     * Método público para alterar o valor do preço
     */
    public void alteraPreco(double preco) {

        // Proteção do atributo...
        // Verifica se o novo valor do Preço é nulo
        // Não pode haver produto com o preço nulo
        if (precoValido(preco))
            this.preco = preco;
    }

    /**
     * Método público para alterar o valor do nome
     */
    public void alteraNome(String nome) {

        // Proteção do atributo...
        // Verifica se o novo valor do nome é vazio
        // Não pode haver produto com o nome vazio
        if (nomeNulo(nome) == false)
            this.nome = nome;
    }
}

```

Note que os métodos **precoValido()** e **nomeNulo()** são declarados como **private** pois são utilizados, respectivamente, pelos métodos **alteraPreco()** e **alteraNome()**. Outra vantagem muito importante do encapsulamento é que, uma vez que o acesso ao método está protegido, podemos alterar o comportamento interno da operação pública (acesso) sem que isso seja de conhecimento de classes externas. Por exemplo, a classe que chamou a operação **alteraPreco()** não precisa necessariamente conhecer os detalhes da implementação e a lógica de programação incluída no método **alteraPreco()**, há um isolamento conveniente entre os papéis de cada classe: se o método **alteraPreco()** for modificado na classe Produto não há a necessidade de nenhuma alteração na classe principal.

1.4 Definindo classes com métodos "get" e "set"

Na linguagem Java, há uma convenção de nomenclatura para os métodos que têm como finalidade acessar ou alterar as propriedades de um objeto. Segundo essa convenção, os nomes dos métodos que permitem a consulta das propriedades de um objeto devem possuir o prefixo **get**. Analogamente, os nomes dos métodos que permitem a alteração das propriedades de um objeto devem possuir o prefixo **set**.

Na maioria dos casos, é muito conveniente seguir essa convenção, pois os desenvolvedores Java já estão acostumados com essas regras de nomenclatura e o funcionamento de muitas bibliotecas do Java depende fortemente desse padrão. **Get** e **Set**, nos exemplos anteriores, correspondem às operações que retornam e modificam, respectivamente, os valores de **todos** os atributos de uma classe. Por exemplo: para definir a classe `Funcionario`, com atributos **nome** e **salario** e com métodos **get** e **set** fazemos:

```
public class Funcionario {

    /**
     * Atributos definidos como privados...
     */
    private String nome;
    private double salario;

    // Construtor público
    public Funcionario(String nome, double salario){

        this.nome = nome;
        this.salario = salario;
    }

    /**
     * @return the nome
     */
    public String getNome() {
        return nome;
    }

    /**
     * @param nome the nome to set
     */
    public void setNome(String nome) {
        this.nome = nome;
    }

    /**
     * @return the salario
     */
}
```

```

    public double getSalario() {
        return salario;
    }

    /**
     * @param salario the salario to set
     */
    public void setSalario(double salario) {
        this.salario = salario;
    }
}

```

O código dos métodos **get** e **set** pode ser gerado automaticamente pelo eclipse. Basta que já se tenham definidos os atributos com seus tipos e visibilidades e, após a definição do construtor, Botão Direito>>Source>>Generate Getters and Setters.

1.5 Atributos e Métodos Estáticos

Conforme definido, classes são modelos para a geração de objetos. Para cada classe, temos um conjunto de atributos e cada objeto pode representar valores diferentes para esses atributos. Da mesma forma, métodos podem alterar objetos de maneira diferente. No entanto, em muitos casos, é necessário que o valor de um atributo de classe seja o mesmo para todos os seus objetos. Uma vez definido tal atributo, se modificado, ele estará modificado em **todos** os objetos da classe. Tal atributo é chamado **atributo de classe** ou **atributo estático**.

Da mesma forma, podemos definir um **método estático** ou **método de classe** como aquele que, além de ser comum a todos os objetos, pode ser acessado diretamente por meio da classe. Geralmente métodos estáticos são os que modificam o valor de atributos estáticos.

Por exemplo: considere a classe **Funcionario**: além dos atributos nome, idade e salario, considere que todos os funcionários recebem um auxilio alimentação no valor de R\$ 300. Dessa forma, **todos** os objetos criados a classe Funcionario deverão ter o campo **valeRefeicao** com o valor de R\$ 300. **valeRefeicao** é um atributo estático ou atributo de classe. Podemos definir uma operação **alteraValeRefeicao()**, um **método estático**. Definiremos então a classe Funcionario como segue:

```

public class Funcionario {

```

```

/**
 * Atributos definidos como privados...
 * */
private String nome;
private double salario;

// Atributo estático: comum a todos os objetos da classe Funcionario
private static double valeRefeicao = 300;

// Construtor público
public Funcionario(String nome, double salario){

    this.nome = nome;
    this.salario = salario;
}

/**
 * @return the nome
 */
public String getNome() {
    return nome;
}

/**
 * @param nome the nome to set
 */
public void setNome(String nome) {
    this.nome = nome;
}

/**
 * @return the salario
 */
public double getSalario() {
    return salario;
}

/**
 * @param salario the salario to set
 */
public void setSalario(double salario) {
    this.salario = salario;
}

/**
 * Método estático alteraValeRefeicao() para
 * alterar o atributo estático
 * */
public static void alteraValeRefeicao(double vale){

    Funcionario.valeRefeicao = vale;
}

/**
 * Método estático alteraValeRefeicao() para
 * alterar o atributo estático
 * */
public static void mostraValeRefeicao(){

    System.out.println("Vale Refeição: " +

```

```
Funcionario.valeRefeicao);  
    }  
}
```

Note que o atributo **valeRefeicao** é definido como **private static**, ou seja, pode ser acessado ou modificado somente por meio dos métodos estáticos correspondentes `alteraValeRefeicao(double vale)` e `static void mostraValeRefeicao()`. Observe ainda que nos dois métodos estáticos podemos fazer referência ao atributo estático chamando inicialmente o nome da classe seguido de um ponto. Na classe de teste teremos o seguinte:

```
public static void main(String[] args) {  
  
    Funcionario f1 = new Funcionario("Maria", 2000);  
    Funcionario f2 = new Funcionario("João", 1000);  
  
    System.out.println("Vale Refeição de f1: ");  
    f1.mostraValeRefeicao();  
    System.out.println("Vale Refeição de f2: ");  
    f2.mostraValeRefeicao();  
  
    Funcionario.alteraValeRefeicao(400);  
  
    System.out.println("Vale Refeição de f1: ");  
    f1.mostraValeRefeicao();  
    System.out.println("Vale Refeição de f2: ");  
    f2.mostraValeRefeicao();  
  
}
```

Na listagem acima temos a criação de dois objetos da classe `Funcionario`, `f1` e `f2`. Ambos compartilham a mesma variável estática e de classe **valeRefeicao**. Os métodos estáticos podem ser acessados tanto por cada objeto (**f1.mostraValeRefeicao()**) quanto pela própria classe. No exemplo acima o valor do vale refeição foi alterado para 400, por meio da chamada de um método estático `Funcionario.alteraValeRefeicao(400)` que alterou o valor para todos os objetos da classe **Funcionario**.

Exercícios da Unidade

1. O que você entende por atributos e métodos privados? Qual a importância de se definir atributos privados para uma classe?
2. O que você entende por encapsulamento? Defina para a classe Conta da unidade anterior encapsulamento para seus atributos **numero**, **cliente** e **saldo**.
3. Crie um projeto no Eclipse com a estrutura da Figura 14. Implemente as classes Livro (com atributos titulo, autor, editora e ano), Empregado (com atributos nome, idade e salario) e Cliente (com atributos nome, idade e endereco). Crie operações **get** e **set** para cada uma das classes. Crie uma classe Principal deverão ser chamados pelo menos uma vez cada um dos métodos **get** e **set** das classes.
4. Construa a classe Aluno com os atributos nome, nota1, nota2 e nota3. Defina as visibilidades corretas para os atributos, o construtor e as operações **get** e **set**. Para cada operação **set**, que deverá modificar o valor da nota, faça um teste para **não aceitar notas negativas**. Crie um metodo com retorno **double calculaMedia()** que deverá calcular e retornar a média das três notas. Crie um método **verificaMedia()** que verifica se a media das notas é maior que 7. Na operação **set** do atributo **nome** defina um teste para não permitir que nomes nulos ("") sejam inseridos. Defina uma classe principal para testar a classe Aluno. Na classe principal deverão ser mostrados o nome, as notas e a média do aluno e se a média do mesmo é ou não maior que 7.
5. Crie uma classe Aluno com um atributo estático **nomeProfessor** = "João". Dessa forma, todos os objetos da classe aluno estarão relacionados ao professor de nome " João" . Crie um método estático na classe **mostraNomeProfessor()** que mostrara **this.nomeProfessor** e que deverá ter o mesmo valor de retorno para todos os objetos. Na classe principal modifique o atributo **nomeProfessor** para " Maria" e execute novamente a chamada ao método **mostraNomeProfessor()**.